Instrucciones de uso de la plantilla

Esta plantilla consiste en un proyecto de CMake ya configurado para C++ y/o CUDA con la posibilidad de utilizar OpenMP y MPI con ejemplos incluidos.

1 Requisitos

- **CMake:** Una instalación reciente de CMake. La plantilla requiere CMake de versión >= 3.20.
- **Compilador:** Un compilador de C++ y/o CUDA según se necesite. La lista de lenguajes necesarios para el proyecto se puede modificar en el archivo `CMakeLists.txt`. En el caso de NVCC, el compilador de CUDA, este viene incluido como parte del kit de herramientas de CUDA (*CUDA Toolkit*).
- **Implementación de OpenMP:** La librería que implementa el estándar OpenMP suele ser proporcionada por el propio compilador, por lo que no hace falta hacer nada para obtenerla. Con tener una versión reciente de un compilador de C++ como Clang, MSVC (*Microsoft Visual C++*) o GCC (*GNU Compiler Collection*) es suficiente. Sin embargo, la versión de OpenMP soportada varía entre compilador y compilador [17].
- **Implementación de MPI:** Las librerías que implementan el estándar de MPI (*Message Passing Interface*) son ajenas a los compiladores, por lo que hay que obtenerlas manualmente. Existen diferentes librerías que implementan el estándar MPI como OpenMPI o Microsoft MPI.

2 Identificación de la tarjeta gráfica

Para poder desarrollar software en CUDA correctamente, es necesario primero conocer ciertas especificaciones de la GPU. Debido a que CUDA es una tecnología propietaria de NVIDIA, la GPU debe ser de NVIDIA.

Buscando el modelo de la GPU en una base de datos de tarjetas gráficas [19][14][18][20] se puede consultar la arquitectura [16, *NVIDIA GPU Architectures*] así como la versión de CUDA [14] que soporta, también conocida como la capacidad de cómputo (*compute capability*). Es necesario conocer esta información para poder conocer qué versiones del kit de herramientas de CUDA son compatibles con la GPU así como qué versiones de Visual Studio y qué depurador de CUDA se pueden utilizar.

- **CUDA Toolkit:** En su versión 12.4 Update 1 (la más reciente a fecha 06/05/2024), el kit de herramientas soporta GPUs desde la arquitectura Maxwell hasta Ada y Visual

Studio 2017, 2019 y 2022. No obstante, los procesadores de GPU Maxwell de primera generación (modelo *GM1xx*, CUDA 5.0) han sido declarados como obsoletos y el soporte para estos será retirado en próximas versiones del kit. En caso de que se disponga de una GPU con soporte obsoleto pero no retirado, se necesitará especificar la capacidad de cómputo al compilador de CUDA para que este genere el código máquina correcto para la GPU instalada en el ordenador [18].

- **Depurador de CUDA:** El depurador de CUDA a utilizar depende del sistema operativo en el que se va a ejecutar el software CUDA:
  - **Windows:** Para depurar software CUDA que se ejecuta en Windows, el depurador depende de Visual Studio. En Windows hay dos depuradores disponibles: el depurador tradicional/heredado (*Legacy CUDA Debugger*) que sólo permite depurar código que ejecutado por la GPU (habitualmente mencionado como *GPU CUDA kernel*) y el depurador moderno (*Next-Gen CUDA Debugger*) que permite depurar código ejecutado tanto por la CPU como por la GPU. En el CUDA Toolkit 12.4 Update 1, el depurador tradicional sólo es compatible con las tarjetas gráficas de arquitectura Maxwell, mientras que el depurador moderno no soporta la arquitectura Maxwell, pero sí todas las que la suceden.
  - **Linux:** Para depurar software CUDA que se ejecuta en Linux, el depurador se basa en GDB (*GNU Debugger*). Este soporta todas las tarjetas gráficas que soporta el kit de herramientas de CUDA, pero para poder depurar código que se ejecuta en GPUs de arquitectura Maxwell es necesario configurar una variable de entorno [15].
- **Visual Studio:** Si se quiere desarrollar software CUDA que se ejecute en Windows, es necesario utilizar Visual Studio. El desarrollo y depuración de software CUDA utilizando Visual Studio depende de una extensión incluida con el kit de herramientas de CUDA llamada *Nsight Visual Studio Edition*. Esta soporta las mismas GPUs y las mismas versiones de Visual Studio que la versión del CUDA Toolkit en la que viene incluida. Sin embargo, el depurador de CUDA tradicional es deshabilitado cuando se utiliza la extensión en Visual Studio 2022. Por ello, **aquellos con una GPU de arquitectura Maxwell que quieran depurar software CUDA que se ejecuta en Windows deberán usar Visual Studio 2019**. Cabe señalar que distintas versiones de Visual Studio pueden estar instaladas al mismo tiempo en el mismo ordenador.

3 Preparación del entorno de desarrollo

3.1 Windows

Debido a las particularidades de cómo funciona Windows, es recomendable reiniciar el ordenador después de completar toda la preparación del entorno de desarrollo.

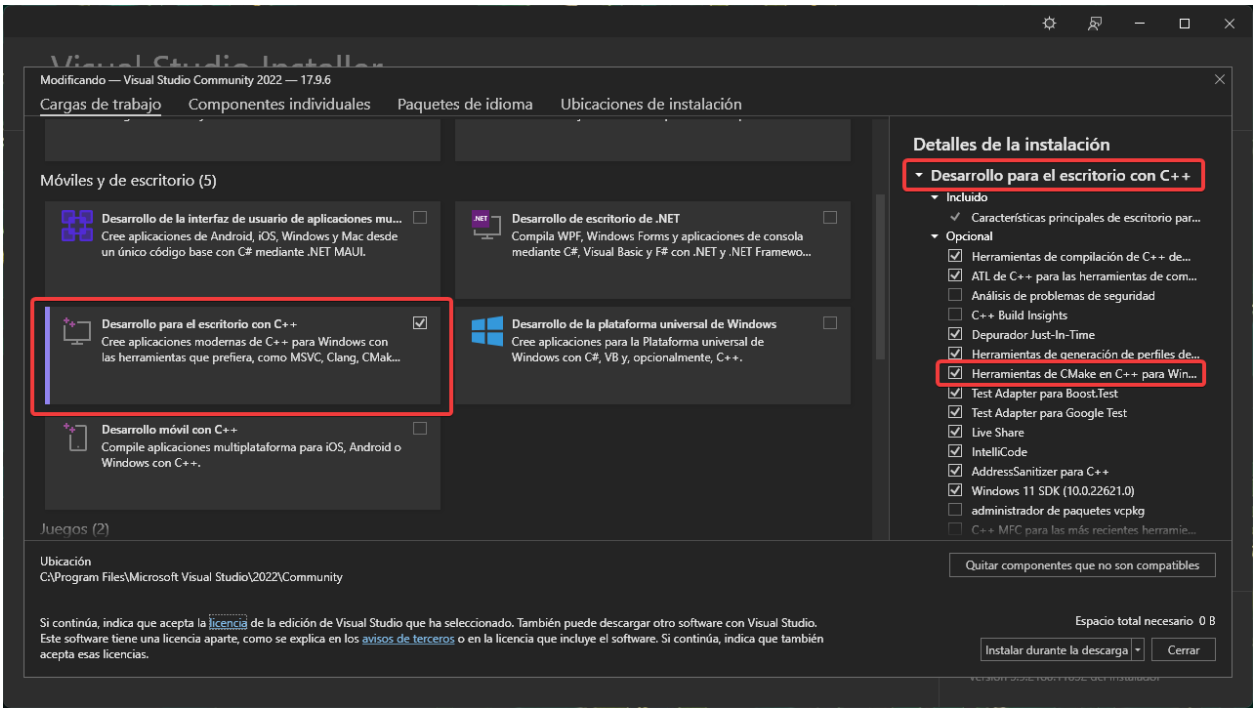
3.1.1 Compilador de C++ y CMake

En Windows el compilador recomendado es MSVC y el CUDA Toolkit depende de él. Este viene incluido al instalar o bien el paquete Build Tools for Visual Studio o bien Visual Studio con la carga de trabajo *Desarrollo para el escritorio con C++*. A pesar de que

Microsoft dispone de una página web oficial dedicada a la descarga de Visual Studio [8], esta sólo permite descargar la versión más reciente de Visual Studio Community, por lo que a continuación se listan varios enlaces directos a los instaladores para varias versiones de Visual Studio Community:

- **Visual Studio 2022:**
  - `https://aka.ms/vs/17/release/vs_community.exe`
  - `https://visualstudio.microsoft.com/es/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2022`
  - `https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&rel=17`
- **Visual Studio 2019:**
  - `https://aka.ms/vs/16/release/vs_community.exe`
  - `https://visualstudio.microsoft.com/es/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2019`
  - `https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&rel=16`
- **Visual Studio 2017:**
  - `https://aka.ms/vs/15/release/vs_community.exe`
  - `https://visualstudio.microsoft.com/es/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2017`
  - `https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&rel=15`

Además, la propia Microsoft distribuye CMake junto con Visual Studio en forma de componente individual, por lo que este se puede añadir directamente desde el instalador de Visual Studio. En caso de ser necesario, la propia desarrolladora de CMake ofrece descargas para Windows [2].



Si Visual Studio ya está instalado en el ordenador, se puede añadir CMake accediendo al instalador de Visual Studio y modificando la instalación.

3.1.2 Kit de herramientas de CUDA

Para programar en CUDA, es necesario contar con una GPU de NVIDIA así como con el kit de herramientas de CUDA (*CUDA Toolkit*), disponible para instalar en la página web de NVIDIA para desarrolladores. Si se tienen drivers de NVIDIA actualizados, en el instalador se puede utilizar la instalación personalizada para omitir la instalación de drivers.

Sin embargo, como el kit de herramientas de CUDA depende de MSVC, será necesario instalarlo también si es que aún no está instalado para poder desarrollar software escrito en CUDA en Windows.

Debido a que el instalador del CUDA Toolkit sólo instala Nsight Visual Studio Edition para aquellas versiones de Visual Studio que estén presentes en el momento de su instalación, **si se han instalado versiones nuevas de Visual Studio después de haber instalado el CUDA Toolkit, el instalador deberá ser ejecutado de nuevo para añadir la extensión a dichas versiones de Visual Studio**. Para no tener que instalar todo de nuevo, en el instalador se puede elegir la instalación personalizada y únicamente marcar para instalación el componente *CUDA > Nsight VSE*.

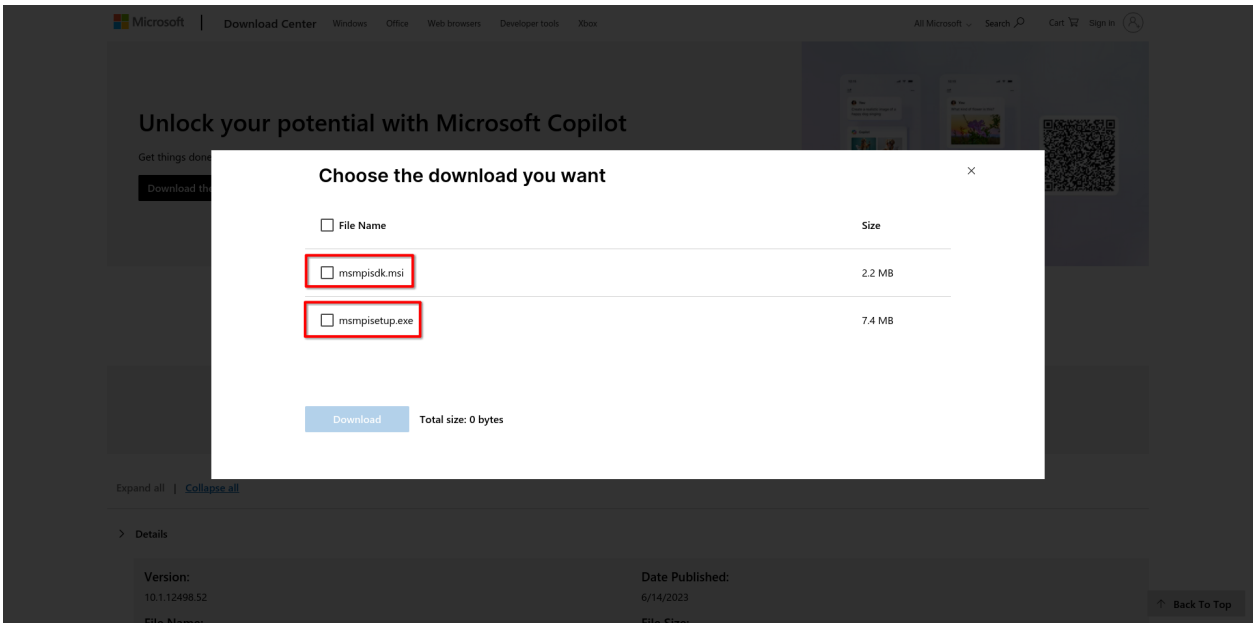
3.1.3 Implementación de MPI

Debido a que el propio órgano que crea el estándar MPI no ofrece una implementación ”oficial”, existen multiples librerías que facilitan el desarrollo de software que use el estándar. Debido a su facilidad de instalación y su integración con el conjunto de herramientas de desarrollo para Windows ofrecido por Microsoft, en esta guía se demostrará cómo instalar Microsoft MPI [10], la implementación del estándar MPI desarrollada por Microsoft para Windows.

Si se está usando Windows 11 o una versión reciente de Windows 10, se pueden instalar tanto la librería como los archivos necesarios para el desarrollo de aplicaciones con Microsoft MPI de forma automatizada utilizando el gestor de paquetes de Windows [12]. Para ello, abrir una terminal, ya sea *Símbolo del sistema* (también conocida como la CMD) o Powershell y ejecutar los siguientes comandos:

```
winget install -e --id Microsoft.msmpi
winget install -e --id Microsoft.msmpisdk
```

Si por algún motivo no se puede utilizar el gestor de paquetes de Windows, ambos paquetes se pueden instalar manualmente desde el enlace proporcionado en la documentación de Microsoft MPI [10, *MS-MPI Downloads*]. Una vez que en el centro de descargas de Microsoft se pida elegir qué se quiere descargar, se debe descargar e instalar tanto el paquete estándar como el SDK.



3.1.4 Windows Terminal

Es conveniente instalar la nueva terminal de Windows, la cuál está disponible en la Microsoft Store [13][9]. Esta cuenta con integración con Visual Studio y permite abrir líneas de comandos para desarrolladores de Visual Studio (*Visual Studio Command Prompt*) directamente desde su interfaz de usuario sin mucha complicación.

3.2 Linux

Debido a que en Linux cada distribución incluye los paquetes de diferentes maneras y versiones, la instalación dependerá de la distribución Linux utilizada.

3.2.1 Compilador de C++ y CMake

En Arch Linux, todos los paquetes necesarios están disponibles desde los repositorios oficiales. En este comando se instalan el compilador de C++ Clang, la implementación de

LLVM de OpenMP, el depurador de LLVM, CMake y un sistema de compilación compatible con CMake. En lugar de Clang también se puede instalar GCC.

```
sudo pacman -Syu clang openmp lldb cmake ninja
```

En Ubuntu 22.04, todos los paquetes necesarios están disponibles desde los repositorios oficiales. En este comando se instalan el compilador de C++ Clang, la implementación de LLVM de OpenMP, el depurador de LLVM, CMake y un sistema de compilación compatible con CMake. En lugar de Clang también se puede instalar GCC.

```
sudo apt install clang libomp-dev lldb cmake ninja-build
```

3.2.2 Kit de herramientas de CUDA

En Arch Linux, tanto el kit de herramientas como las herramientas Nsight (`cuda-tools`) están disponibles desde los repositorios oficiales. En el siguiente comando, instalamos el kit de herramientas y el depurador GDB, el cuál es necesario para poder usar el depurador de CUDA en Linux.

```
sudo pacman -Syu cuda gdb
```

En Ubuntu 22.04, tanto el kit de herramientas como las herramientas Nsight (`nsight-*`) están disponibles desde los repositorios oficiales. En el siguiente comando, instalamos el kit de herramientas y el depurador de CUDA.

```
sudo apt install nvidia-cuda-toolkit nvidia-cuda-gdb
```

3.2.3 Implementación de MPI

En Arch Linux, OpenMPI está disponible desde los repositorios oficiales.

```
sudo pacman -Syu openmpi
```

En Ubuntu 22.04, OpenMPI está disponible desde los repositorios oficiales.

```
sudo apt install libopenmpi-dev
```

4 Utilización de la plantilla

Tal y como se ha mencionado anteriormente, la plantilla se basa en una descripción de proyecto para CMake, el cuál puede ser usado directamente desde la terminal o integrado con varios programas de desarrollo como Visual Studio, Visual Studio Code o CLion.

Debido a problemas de compatibilidad del depurador de CUDA con rutas de archivo que contienen caracteres especiales, para poder usar el depurador CUDA es necesario extraer y usar la plantilla en una carpeta cuya ruta no contenga caracteres especiales. Esto es de especial relevancia para aquellas personas familiarizadas con lenguas no occidentales, ya que a menudo el nombre de su carpeta de usuario contiene caracteres fuera de los reconocidos por el estándar ASCII.

4.1 Estructura del proyecto

El proyecto consiste en varios archivos y carpetas cada uno con una función concreta. Respecto a los archivos que describen el proyecto (`CMakeLists.txt`), estos están comentados de forma extensiva con enlaces de ayuda y es recomendable leerlos, aunque sea por encima, para hacerse una idea de qué es lo que hacen.

- `CMakeLists.txt`: Es el archivo principal de descripción del proyecto y el primero que CMake lee cuando se le invoca.
- `main.cpp`: Es el archivo que contiene el `main()` del ejecutable `programa`. Este ejecutable es el que representa el programa que queréis crear usando la plantilla. La declaración de dicho ejecutable está al final del archivo `CMakeLists.txt` y puede ser modificada como sea oportuno para, entre otras cosas, añadir archivos de código fuente ya sean de C++ (`.cpp`/`.hpp`) o CUDA (`.cu`/`.cuh`) o vincular el ejecutable con librerías.
- `examples`: Esta carpeta contiene varios ejemplos de ejecutables CUDA y C++, cada uno con su correspondiente descripción en lenguaje CMake. Es recomendable leerlos para saber cómo utilizar las librerías de OpenMP y MPI desde CMake. El archivo `examples/CMakeLists.txt` es cargado por el `CMakeLists.txt` principal utilizando el comando `add_subdirectory()` y se encarga de añadir los subdirectorios de cada ejemplo dependiendo de qué lenguajes han sido habilitados en la declaración del proyecto.
  - `cpp_example`: Un ejecutable simple de C++ imprimiendo el clásico ”hola mundo”.
  - `cuda_cpp_example`: Un ejecutable que emplea código escrito usando C++ y CUDA que imprime el clásico ”hola mundo” tanto desde la CPU como desde la GPU (usando la CUDA Runtime API) y demuestra cómo llamar a código escrito en CUDA desde C++.
  - `cuda_example`: Un simple ejecutable escrito en CUDA (usando la CUDA Runtime API) que imprime el clásico ”hola mundo” tanto desde la CPU como desde la GPU (usando la CUDA Runtime API).
  - `mpi_example`: Un simple ejecutable escrito en C++ que usa la librería MPI para imprimir un ”hola mundo” por cada hilo de ejecución de la CPU.
  - `openmp_example`: Un simple ejecutable escrito en C++ que usa la librería OpenMP para imprimir un ”hola mundo” por cada hilo de ejecución de la CPU.

Una vez se hayan consultado y probado los ejecutables de interés, es conveniente eliminar la llamada a `add_subdirectory(examples)` escrita al final del `CMakeLists.txt` principal así como la carpeta `examples` para reducir el proyecto a tan solo el ejecutable que se quiere desarrollar. Eso sí, conservando una copia de la plantilla para poder volver a consultarlos siempre que sea necesario.

4.2 Configuración del proyecto

A la hora de configurar el proyecto desde cero, CMake lee la variable de entorno `CUDAARCHS` con el fin de consultar la lista de versiones de CUDA para las que compilar el código CUDA (si es el lenguaje ha sido habilitado en la descripción del proyecto). Por ello, antes de nada es recomendable configurar la lista de arquitecturas para incluir la arquitectura de la GPU del ordenador así como su arquitectura virtual para asegurar que la GPU podrá ejecutar el código compilado.

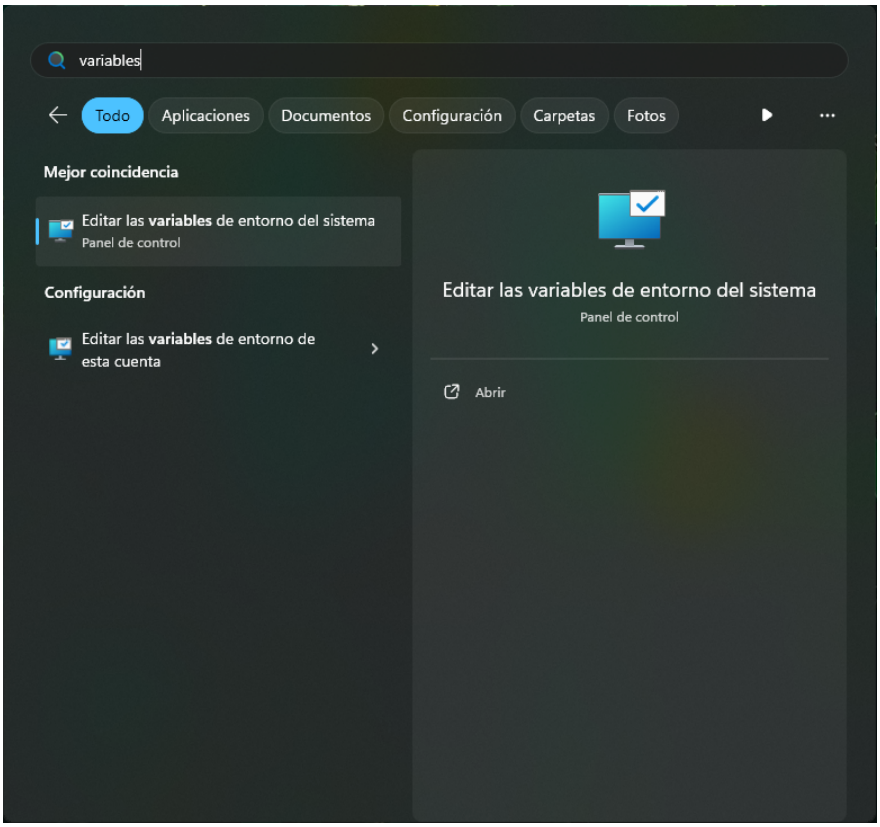
Listar una arquitectura real hace que el compilador genere el código SASS (*Streaming Assembler*) especifico para dicha versión de CUDA, mientras que listar una arquitectura virtual hace que el archivo final incluya el código para la máquina virtual PTX (*Parallel Thread Execution*), el cuál permite que cualquier GPU compatible con una versión de CUDA igual o superior a la especificada pueda ejecutar el software convirtiendo el código PTX a su código SASS específico justo en el momento de ejecutar el código mediante compilación sobre la marcha (*Just-In-Time compilation*) [1][5][4].

Debido a esto, si el profesor pide ejecutables compilados es aconsejable preguntar por qué GPU va a ser usada para probar el código de manera que el ejecutable a entregar haya sido compilado para una versión de CUDA compatible con dicha tarjeta gráfica utilizando CMake directamente desde una terminal y con el argumento `-DCMAKE_CUDA_ARCHITECTURES` para especificar el valor de la variable `CMAKE_CUDA_ARCHITECTURES`.

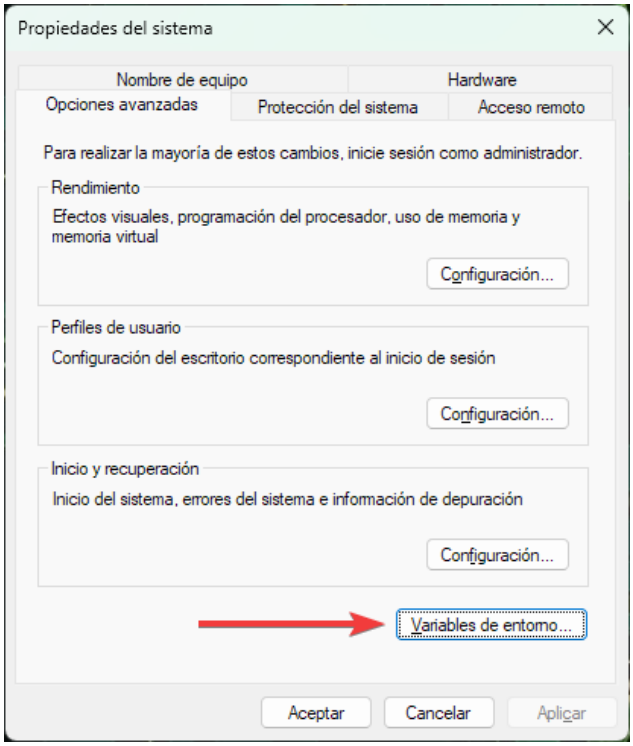
Por ejemplo, para la GPU NVIDIA GeForce RTX 2060, la cual soporta CUDA 7.5, el valor de `CUDAARCHS` debe ser `75` o `75-real;75-virtual` (ambos valores son equivalentes). De esta manera, el programa no sólo podrá ser ejecutado por cualquier GPU con soporte para CUDA 7.5 sino también po cualquier GPU que soporte una versión superior.

En Windows, las variables de entorno se configuran mediante una opción un tanto oculta dentro del sistema operativo. Para ello:

1. Buscar en el menú inicio *Editar las variables de entorno del sistema*:

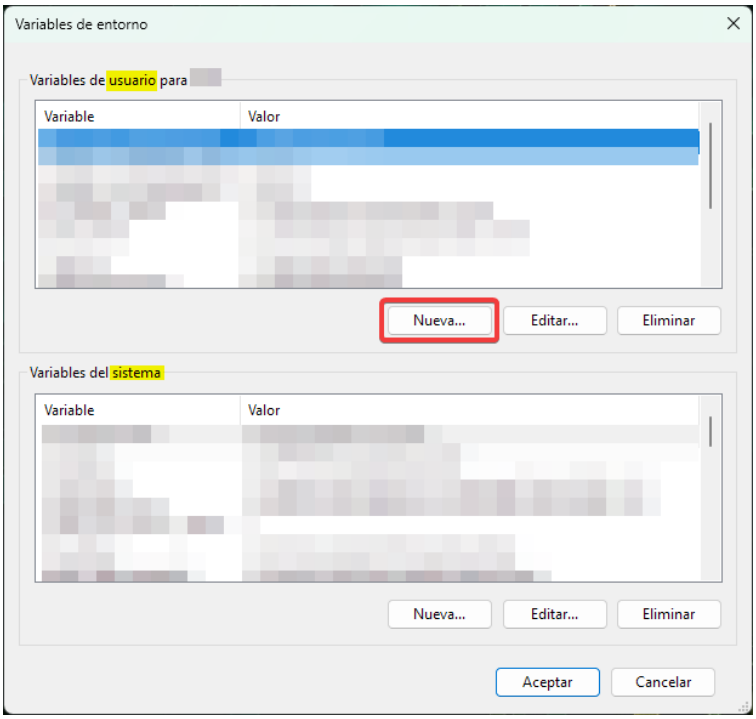


2. En la ventana de propiedades que se abre, hacer clic en la opción *Variables de entorno*:



3. En la ventana de variables de entorno, crear la nueva variable de entorno de usuario

de nombre `CUDAARCHS` y de valor la arquitectura correspondiente a la tarjeta gráfica del ordenador:

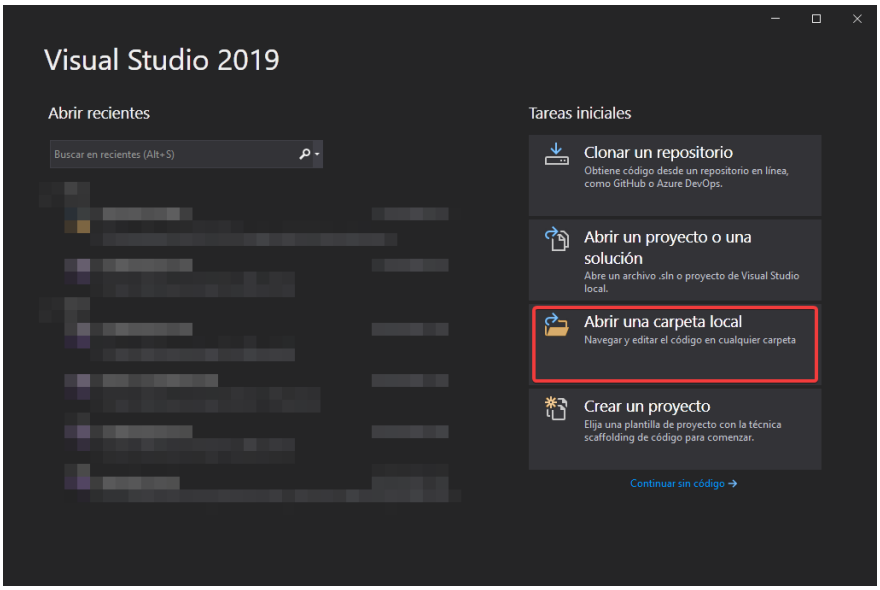


4.2.1 Visual Studio

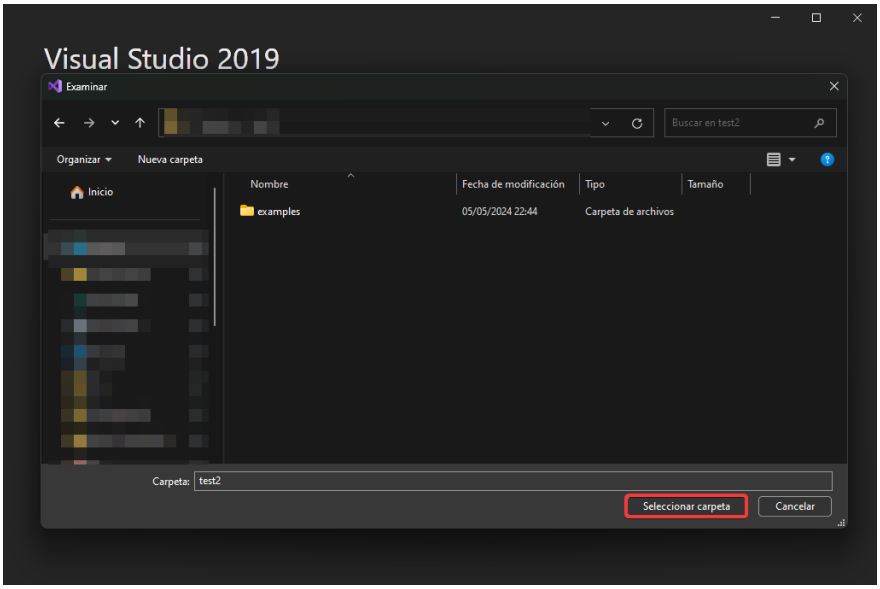
Con el componente correspondiente de Visual Studio instalado, Visual Studio dispone de integración directa con CMake. Por ello, todo lo que hace falta para poder utilizar la plantilla es acceder a Visual Studio y abrir la carpeta en la que se sitúa el archivo `CMakeLists.txt` principal utilizando la opción correspondiente en la pantalla de bienvenida de Visual Studio.

Nota: El proyecto necesita usar la versión CMake 3.20 o superior, por lo que si se que use Visual Studio 2017 o no se tiene una instalación de Visual Studio actualizada se deberá, o bien actualizarla, o instalar la versión más reciente de CMake desde la página oficial [2] y configurar Visual Studio para que use la versión externa [11].

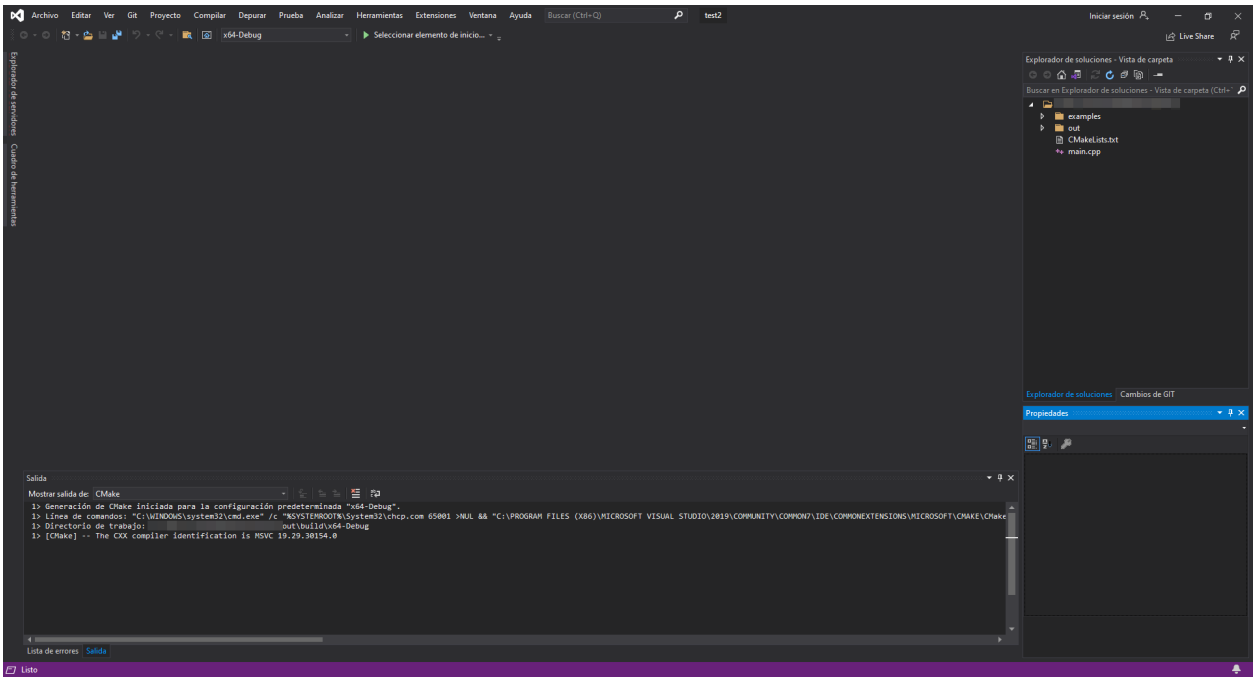
1. Ejecutar Visual Studio. En la pantalla de bienvenida, escoger abrir una carpeta.



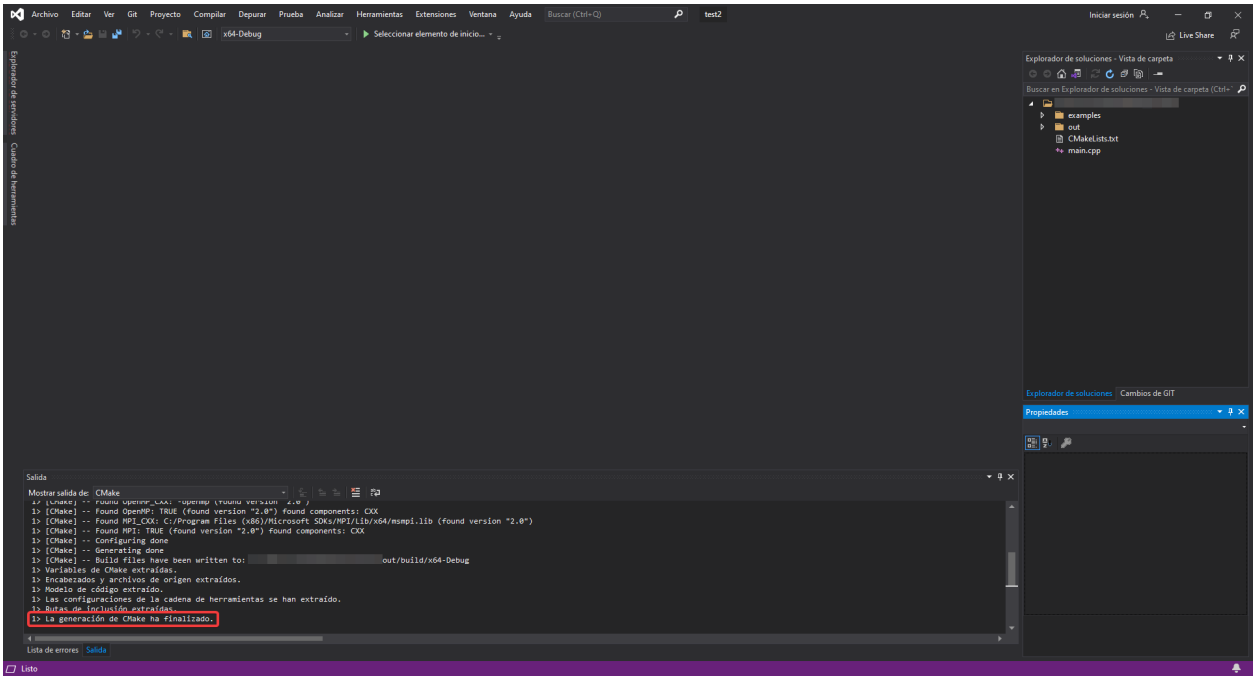
2. Abrir la carpeta con el archivo `CMakeLists.txt` principal.



3. Esperar unos segundos a que Visual Studio detecte que la carpeta abierta contiene un proyecto CMake y empiece a utilizar CMake para configurar el proyecto.



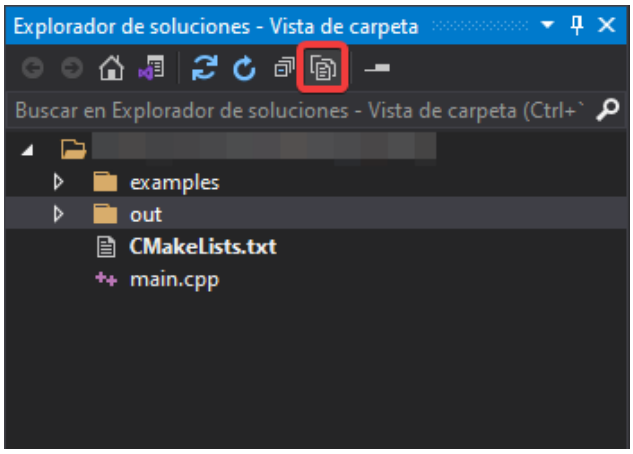
4. Esperar hasta que la configuración del proyecto haya completado. Los directorios resultantes de configurar el proyecto de CMake son ubicados por Visual Studio en la carpeta `out/build`. Cuando se quiera entregar un ejecutable, es conveniente utilizar CMake desde la terminal, tal y como se explica en la sección 4.2.2.



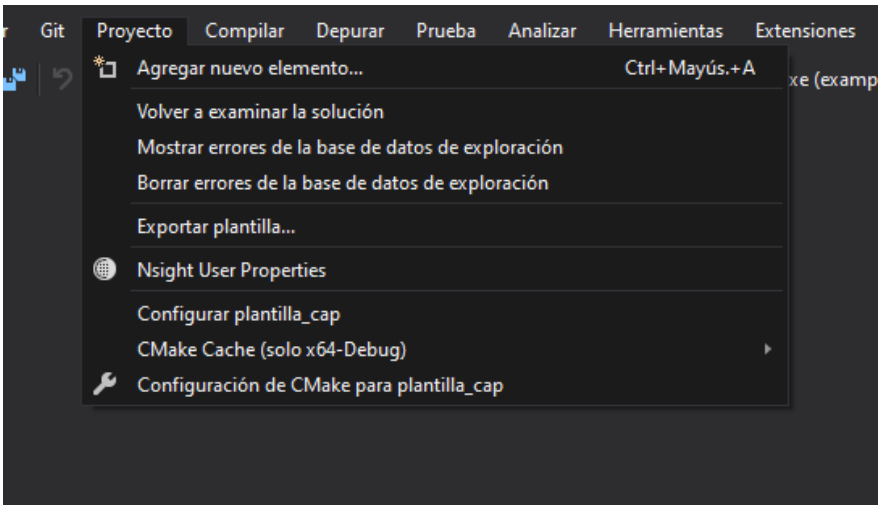
A partir de aquí, ya se puede empezar utilizar Visual Studio de forma normal, con la única excepción de que para los archivos CUDA puede que IntelliSense (el motor de análisis de código de Visual Studio) reporte falsos errores en algunas cosas como la sintaxis de llamada

a kernels CUDA <<<>>>, los cuales no afectan a la compilación del proyecto en ningún momento.

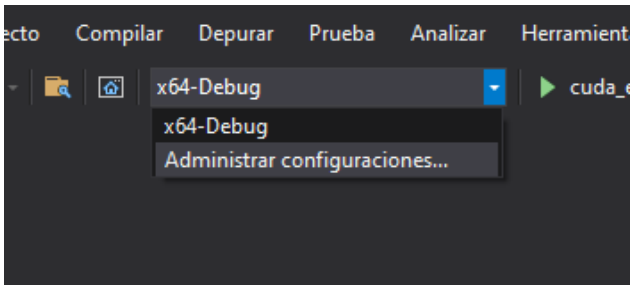
Cuando se trabaja con proyectos CMake, es aconsejable cambiar a la vista de todos los archivos en el explorador de soluciones de Visual Studio.



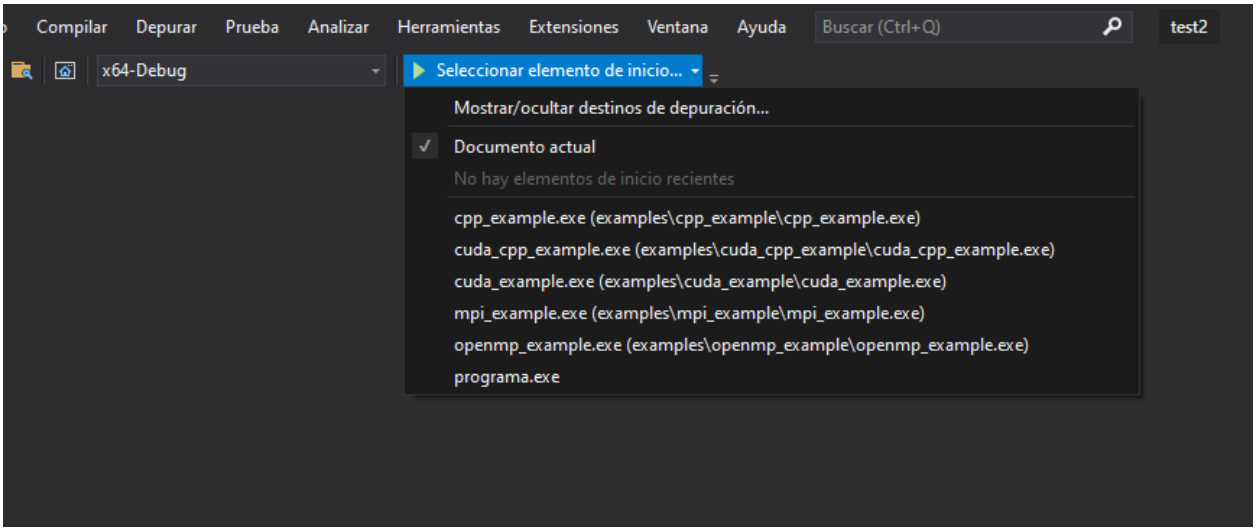
En el menú *Proyecto* se puede acceder a la configuración relacionada con el proyecto así como a las *Nsight User Properties* del elemento de inicio seleccionado. La opción para acceder a las *Nsight User Properties* sólo está disponible cuando hay un elemento de inicio seleccionado.



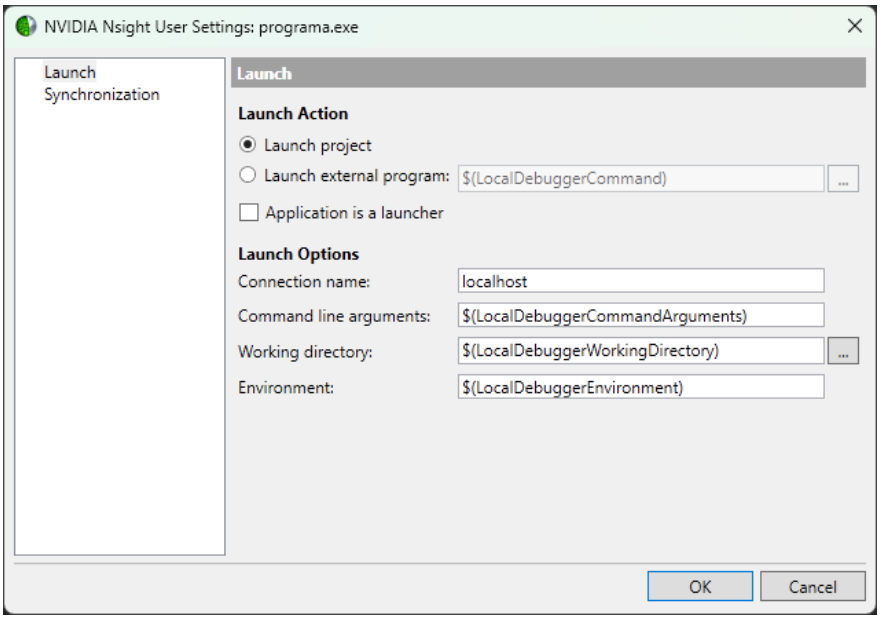
En la parte superior de la pantalla, al lado del botón de depuración, está la lista de configuraciones del proyecto que Visual Studio genera a raíz de llamar a CMake para configurar el proyecto. Esta se crea en base al archivo de Visual Studio `CMakeSettings.json` [11], el cual, si no esta presente y no tampoco figura un archivo `CMakePresets.json` [3][7], Visual Studio muestra una sola configuración `<arch>-Debug` para crear ejecutables aptos para depurar correspondiente a la arquitectura del ordenador.



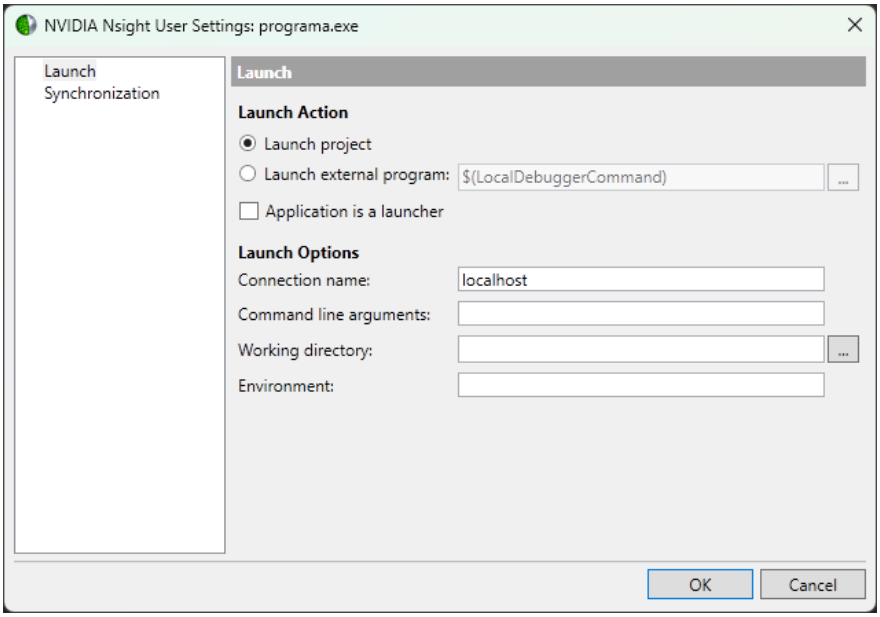
En el menú desplegable del botón de depuración se puede elegir qué elemento de inicio (ejecutable) se quiere ejecutar y depurar.



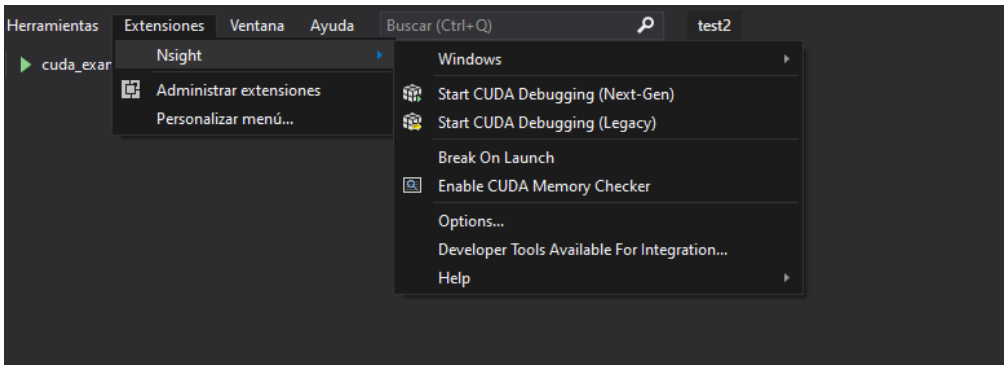
Para depurar código que se ejecuta en la GPU, en *Proyecto > Nsight User Properties* se pueden ver las opciones de ejecución del depurador de CUDA para el elemento de inicio actualmente seleccionado como los argumentos que deben ser pasados al programa al ejecutarlo, el directorio de trabajo o las variables de entorno que debe tener el entorno del ejecutable al depurar utilizando el depurador de CUDA.



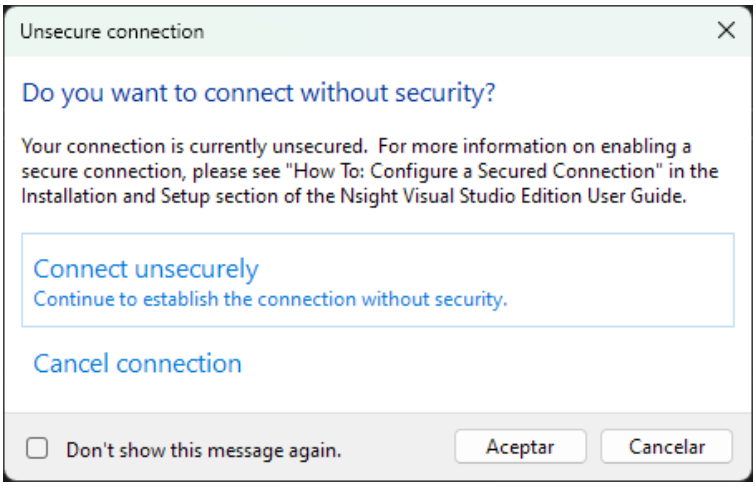
Estas opciones están configuradas por defecto para extraer sus valores del proyecto de Visual Studio utilizando macros de MSBuild [6]. Sin embargo, como no estamos utilizando un proyecto de Visual Studio sino uno de CMake, estas macros no resultan en nada. Antes de poder utilizar el depurador de CUDA con dicho elemento de inicio, sus valores deben ser modificados o bien dejándolas vacías o bien especificando los valores literales deseados.



En el menú *Extensiones* > *Nsight* se sitúan las opciones relacionadas con el depurador de CUDA. Las opciones para iniciar la depuración CUDA sólo son accesibles cuando hay un elemento de inicio seleccionado. En *Extensiones* > *Nsight* > *Options...* se pueden configurar las opciones generales del depurador de CUDA.



Cuando se usa el depurador de CUDA, este se conecta mediante una conexión de red al monitor de Nsight (*Nsight Monitor*), el software que corre en segundo plano como uno de los componentes que permiten la depuración de aplicaciones CUDA y que se encarga de acoplarse al programa que se quiere depurar y vigilar su ejecución. Esta conexión puede ser cifrada o no, y el monitor muestra una advertencia cuando la conexión no está cifrada. A no ser que se esté depurando software que se ejecuta en un ordenador distinto al que estamos usando para depurar, el cifrado de la conexión no tiene especial relevancia y la advertencia puede ser omitida indicando que no se quiere volver a ver el mensaje (*Don't show this message again*) y eligiendo conectarse de forma insegura (*Connect unsecurely*).



4.2.2 Desde una terminal

Nota: En Windows, es necesario hacer estos pasos desde una línea de comandos para desarrolladores de Visual Studio.

1. Abrir la terminal en la misma carpeta en la que se ha extraído la plantilla y se sitúa el archivo `CMakeLists.txt` principal.

2. Llamar a CMake para configurar el proyecto

```
cmake -S . -B build
```

3. Para compilar:

```
cmake --build build
```

Dependiendo de qué generador ha escogido CMake para generar el proyecto, esto genera un ejecutable o bien directamente dentro de la carpeta `build` o dentro de una carpeta `build/<variante>`, donde la variante puede ser `Debug`, `RelWithDebInfo` o `Release`. Si no ha sido especificada, la variante por defecto es `Debug`.

4. Para entregar un ejecutable en caso de que haga falta, se debe borrar la carpeta `build` por completo y volver a configurar el proyecto, esta vez para generar un ejecutable apto para uso fuera del contexto de desarrollo y con las opciones adecuadas.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=<lista>
cmake --build build --config Release
cmake --install build --prefix ./export
```

Al ejecutar estos tres comandos, se exportará el ejecutable de variante `Release` en la carpeta `export`.

Referencias

[1] Kaixi Hou. *CUDA Tips: nvcc's -code, -arch, -gencode*. URL: https://kaixih.github.io/nvcc-options/ (visitado 06-05-2024).

[2] Kitware. *CMake Download*. URL: https://cmake.org/download/ (visitado 06-05-2024).

[3] Kitware, ed. *CMake presets*. URL: https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html (visitado 08-05-2024).

[4] Kitware, ed. *CUDA_ARCHITECTURES - CMake Documentation*. URL: https://cmake.org/cmake/help/latest/prop_tgt/CUDA_ARCHITECTURES.html (visitado 06-05-2024).

[5] Kitware, ed. *CUDAARCHS - CMake Documentation*. URL: https://cmake.org/cmake/help/latest/envvar/CUDAARCHS.html (visitado 06-05-2024).

[6] Microsoft, ed. *Common macros for MSBuild commands and properties*. URL: https://learn.microsoft.com/en-us/cpp/build/reference/common-macros-for-build-commands-and-properties (visitado 08-05-2024).

[7] Microsoft, ed. *Configure and build with CMake Presets in Visual Studio*. URL: https://learn.microsoft.com/en-us/cpp/build/cmake-presets-vs (visitado 08-05-2024).

[8] Microsoft. *Descargas de Visual Studio*. URL: https://visualstudio.microsoft.com/es/downloads/ (visitado 06-05-2024).

[9] Microsoft, ed. *Introducción a Windows Terminal*. URL: https://learn.microsoft.com/es-es/windows/terminal/ (visitado 07-05-2024).

[10] Microsoft. *Microsoft MPI*. URL: https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi (visitado 01-05-2024).

[11] Microsoft, ed. *Personalización de la configuración de compilación de CMake*. URL: https://learn.microsoft.com/es-es/cpp/build/customize-cmake-settings (visitado 06-05-2024).

[12] Microsoft, ed. *Uso de la herramienta winget para instalar y administrar aplicaciones*. URL: https://learn.microsoft.com/es-es/windows/package-manager/winget/ (visitado 01-05-2024).

[13] Microsoft. *Windows Terminal - Aplicaciones de Microsoft*. URL: https://apps.microsoft.com/detail/9n0dx20hk701 (visitado 07-05-2024).

[14] NVIDIA, ed. *CUDA GPUs - Compute Capability*. URL: https://developer.nvidia.com/cuda-gpus (visitado 06-05-2024).

[15] NVIDIA, ed. *CUDA-GDB - Switching to Classic Debugger Backend*. URL: https://docs.nvidia.com/cuda/cuda-gdb/index.html?#switching-to-classic-debugger-backend (visitado 06-05-2024).

[16] NVIDIA, ed. *Key Technologies*. URL: https://developer.nvidia.com/key-technologies (visitado 06-05-2024).

[17] OpenMP, ed. *OpenMP Compilers & Tools*. URL: https://www.openmp.org/resources/openmp-compilers-tools/ (visitado 01-05-2024).

[18] Arnon Shimoni. *Matching CUDA arch and CUDA gencode for various NVIDIA architectures*. URL: https://arnon.dk/matching-sm-architectures-arch-and-gencode-for-various-nvidia-cards/ (visitado 06-05-2024).

[19] TechPowerUp, ed. *GPU Specs Database*. URL: https://www.techpowerup.com/gpu-specs/ (visitado 06-05-2024).

[20] Wikipedia, ed. *List of Nvidia graphics processing units*. URL: https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units (visitado 06-05-2024).